

SE-LAB3

Hostel Maintenance & Issue Ticketing System

Name: Deeptanshu Kumar SRN: PES1UG24AM357 Class/Sec: 5-F

Architecture Selection: We chose Layered Architecture for the Hostel Maintenance & Issue Ticketing System.

Reason 1 — Clear separation of concerns across distinct user roles

The system serves two very different front-end users — residents (ticket reporting, status tracking, closure sign-off) and the Maintenance Warden (assignment, dashboard, SLA monitoring) — but both rely on the same underlying business rules (ticket lifecycle, staff-category matching, SLA thresholds). A layered structure lets the Presentation Layer vary independently per role while the Business Layer enforces these rules exactly once, avoiding duplicated or inconsistent logic between the resident app and the warden dashboard.

Reason 2 — Workload and scale fit a single-institution deployment

This is a single-hostel system with a bounded, predictable number of concurrent residents and wardens rather than an internet-scale consumer product. The ticket workflow (Open → Assigned → In Progress → Resolved → Closed) is well-understood and changes together as one release. A layered monolith delivers this without the deployment, versioning and network-latency overhead that a microservices split would add, keeping the system easy for a small team to build, test and maintain.

Security Advantage

Because only the Business Layer is permitted to call the Data Layer (TicketDataInterface, StaffDataInterface), all authentication, authorization, and validation — e.g. confirming a resident owns a ticket before allowing closure sign-off (FR-004), or that a warden may only assign staff within a matched category (FR-002) — is enforced at a single, well-defined boundary. The Presentation Layer and the external Notification Service never touch the database directly, which shrinks the attack surface compared to an architecture where every service independently manages its own data access.

Performance Benefit

NFR-002 requires ticket submission with a compressed photo to complete in under 5 seconds on a standard 4G connection. Because the Presentation, Business and Data Layers run as in-process calls within one deployable application rather than as independently deployed services communicating over the network, ticket creation avoids the extra network hops and serialization overhead that a microservices split would introduce — helping keep the 95th-percentile submission time under the required threshold.

Component Diagram

Hostel Maintenance & Issue Ticketing System — Layered Architecture (Component Diagram)

